Listing 4.8 An example of using DuckDB and SQL to directly query data stored in a file in Parquet format with needing to load the data into a database first.

```
import duckdb, requests, time

url = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_
➥2024-01.parquet"
with open('taxi.parquet', 'wb') as f: #1
  f.write(requests.get(url).content)

start_time = time.time()
result = duckdb.query(""" #2
  SELECT
      COUNT(*) as trips,
      ROUND(AVG(fare_amount), 2) as avg_fare,
      ROUND(MIN(fare_amount), 2) as min_fare,
      ROUND(MAX(fare_amount), 2) as max_fare
  FROM 'taxi.parquet'
""").fetchone()
query_time = time.time() - start_time

print(f"\nResults: {result[0]:,} trips, avg ${result[1]},
➥range ${result[2]}-${result[3]}")
```

#1 Get a Parquet file containing taxi trip data from January 2024.
#2 Use DuckDB to query the Parquet file using standard SQL.

4.5.3 Data lakes and lakehouse architectures

While columnar analytical storage excels for structured data, digital twins generate diverse formats—JSON sensor logs, images, PDFs, and ML model outputs—that don’t fit predefined schemas. Traditional analytical databases struggle with this heterogeneity, driving organizations toward data lake architectures. Data lakes shift from "schema-on-write" to "schema-on-read" approaches, preserving raw data in native formats and applying structure only when needed for analysis. This provides flexibility for diverse data types while maintaining cost-effective storage.

Traditional data lakes store diverse data effectively but lack performance and ACID transaction support required for analytical applications. Lakehouse architectures address these gaps by adding database-like capabilities to object storage, combining data lake flexibility with data warehouse performance and reliability. Data lakes often store data in an optimized columnar file format such as Apache Parquet, in an object store like S3, but to modify these files requires expensive read, modify, write cycles. Lakehouse storage technologies enable data lakes, using low cost and virtually unlimited object storage, to behave like data warehouses, supporting transactions such as merge and upsert.